



Multiple Horizon Ensemble Method

**a novel framework designed for
multi-step time series forecasting**

Presented by Bortolussi, Bredariol, Savorgnan

Our outline

Theoretical Study

about multi-step
time series forecasting

Implementation

of the novel MHEMe
and experimentations

Evaluation

on 5 different
benchmarks

Intro to Time Series Forecasting

■ **Basic
concepts**

■ **Statistical
models**

■ **ML
models**

■ **Ensemble
methods**

01

Time Series

The Basic Concepts

Time series

$$\{x_0, x_1, \dots, x_t\}$$

Stationarity VS

Trend & Seasonality

$$P[x_0, x_1, \dots, x_t] = P[x_h, x_{1+h}, \dots, x_{t+h}]$$

Autocorrelation

$$\rho_{x_s x_t} = \frac{\mathbb{C}[x_s, x_t]}{\sqrt{\mathbb{V}[x_s] \mathbb{V}[x_t]}}$$

Partial Autocorrelation

$$\rho_{sy|z} = \frac{\rho_{sy} - \rho_{sz}\rho_{yz}}{\sqrt{(1 - \rho_{sz}^2)(1 - \rho_{yz}^2)}}$$

$$s = x_t$$

$$y = x_{t-k}$$

$$z = x_{t-1}, x_{t-2}, \dots, x_{t-k+1}$$

Statistical models

SARIMA framework

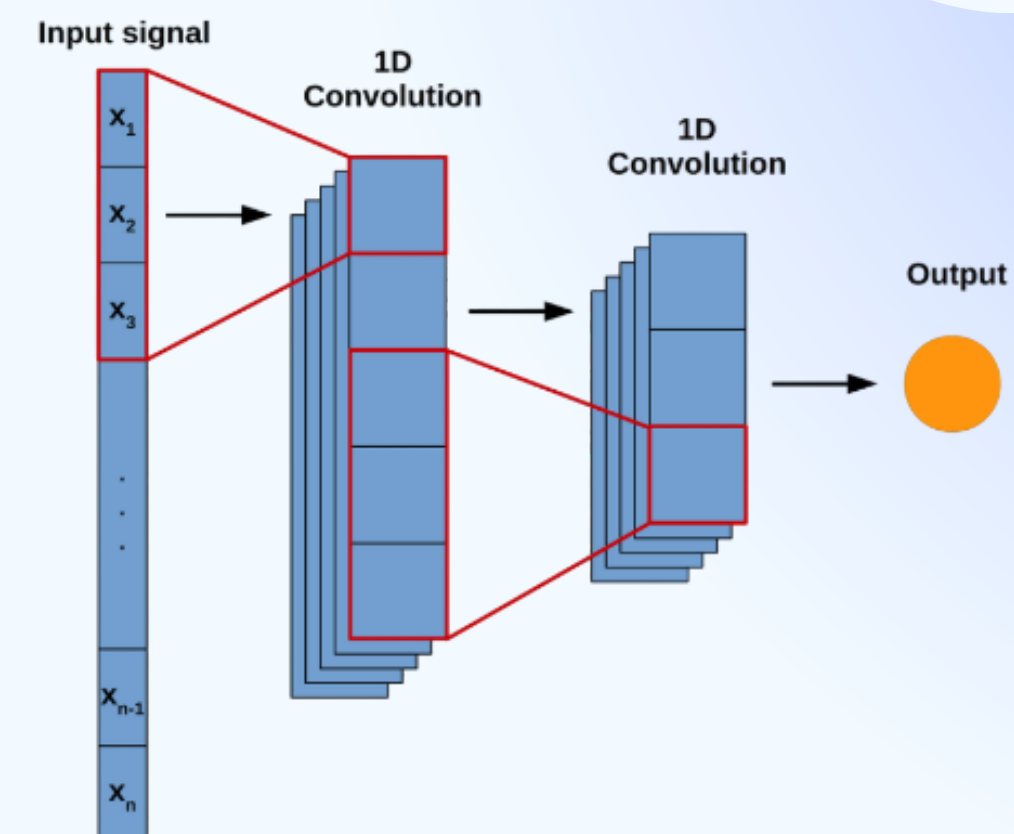
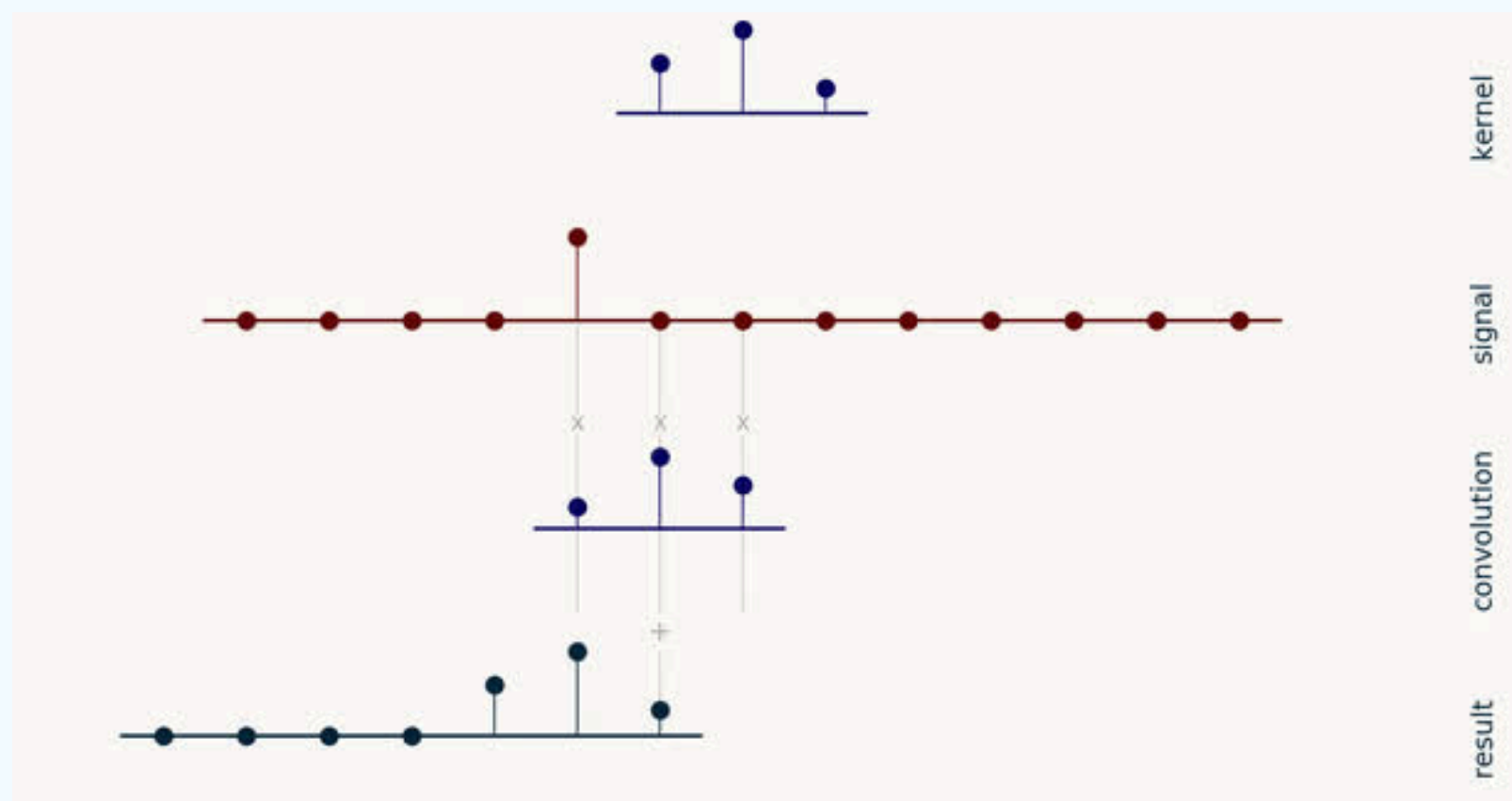
$$\phi_p(B)(1-B)^d\Phi_P(B^s)(1-B^s)^Dy_i = \theta_q(B)\Theta_Q(B^s)\varepsilon_i$$

Seasonal	Data present repeated patterns
Auto-Regressive	Data derives from prior steps
Integrated	Stationarity enforced via data differences
Moving-Average	Forecast error depends <i>linearly</i> on prior errors

Machine-Learning Models

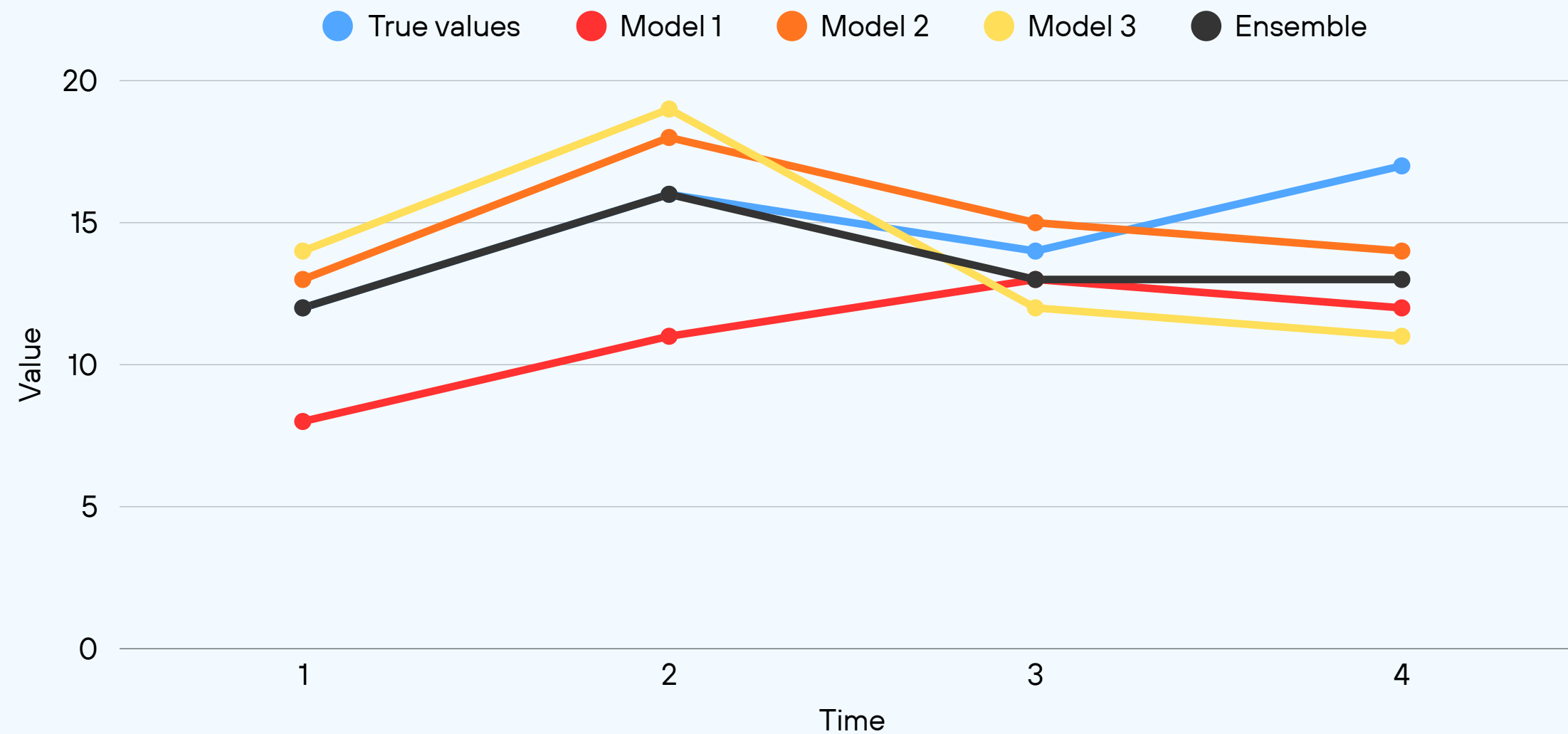
Temporal Convolutional Networks

01



Ensemble Methods

For more robust forecastings



- **Model choice**
- **Weighting Strategy**
 - Uniform
 - Error-based

Univariate MHEMe

■ Framework
formulation

■ Base-model
choice

■ Horizon-
aware loss

■ Weights
computation

02

Framework formulation

- the idea

The Univariate MHEMe is a time series forecasting technique designed to combine the advantages of direct forecasting, autoregressive modeling, and variance-based ensemble methods

Direct models

Direct forecasting methods produce multi-step predictions up to a fixed horizon

Autoregressive modeling

Autoregressive forecasting enables prediction over an unbounded horizon

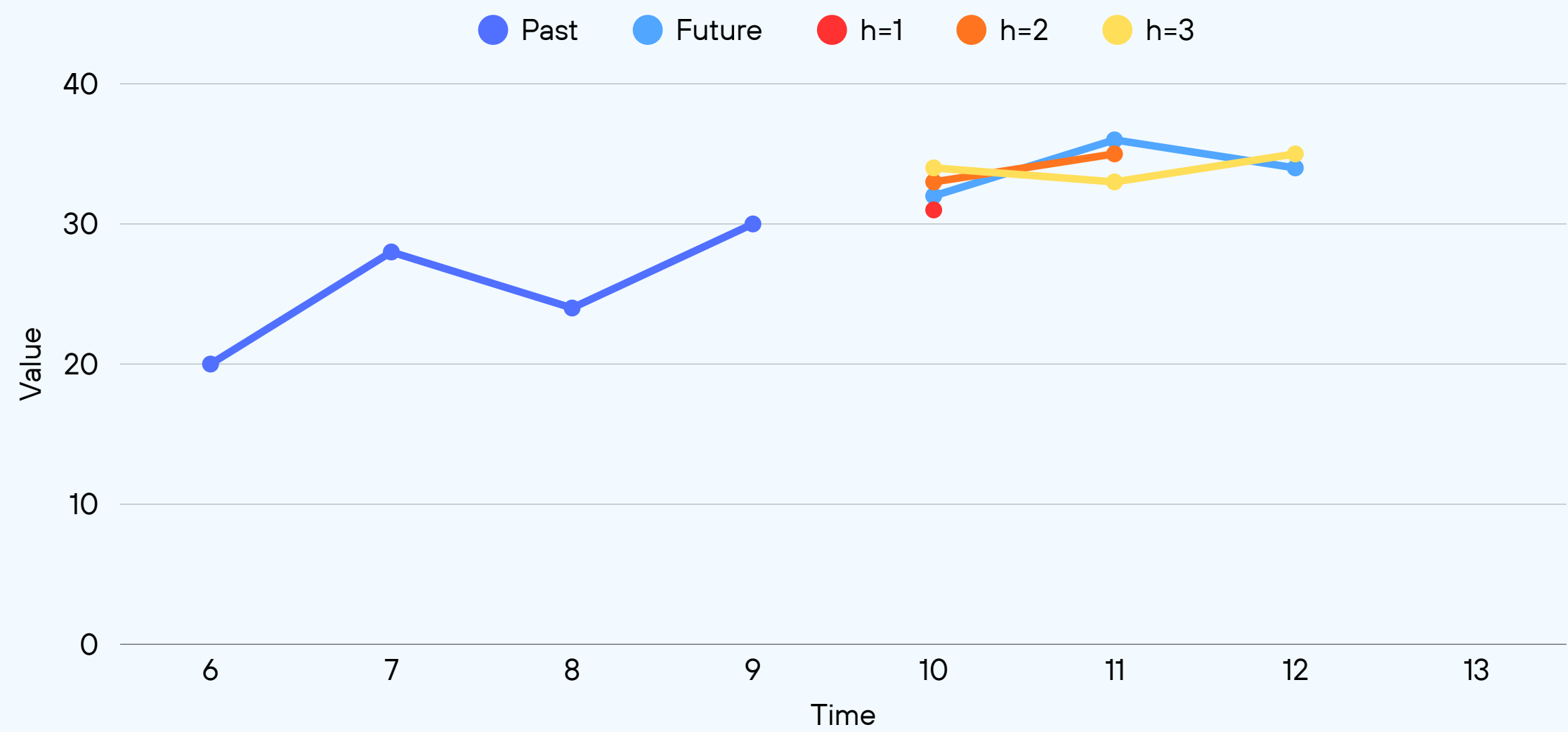
Ensemble methods

Ensemble methods let to combine the results into a single final prediction



Step 1

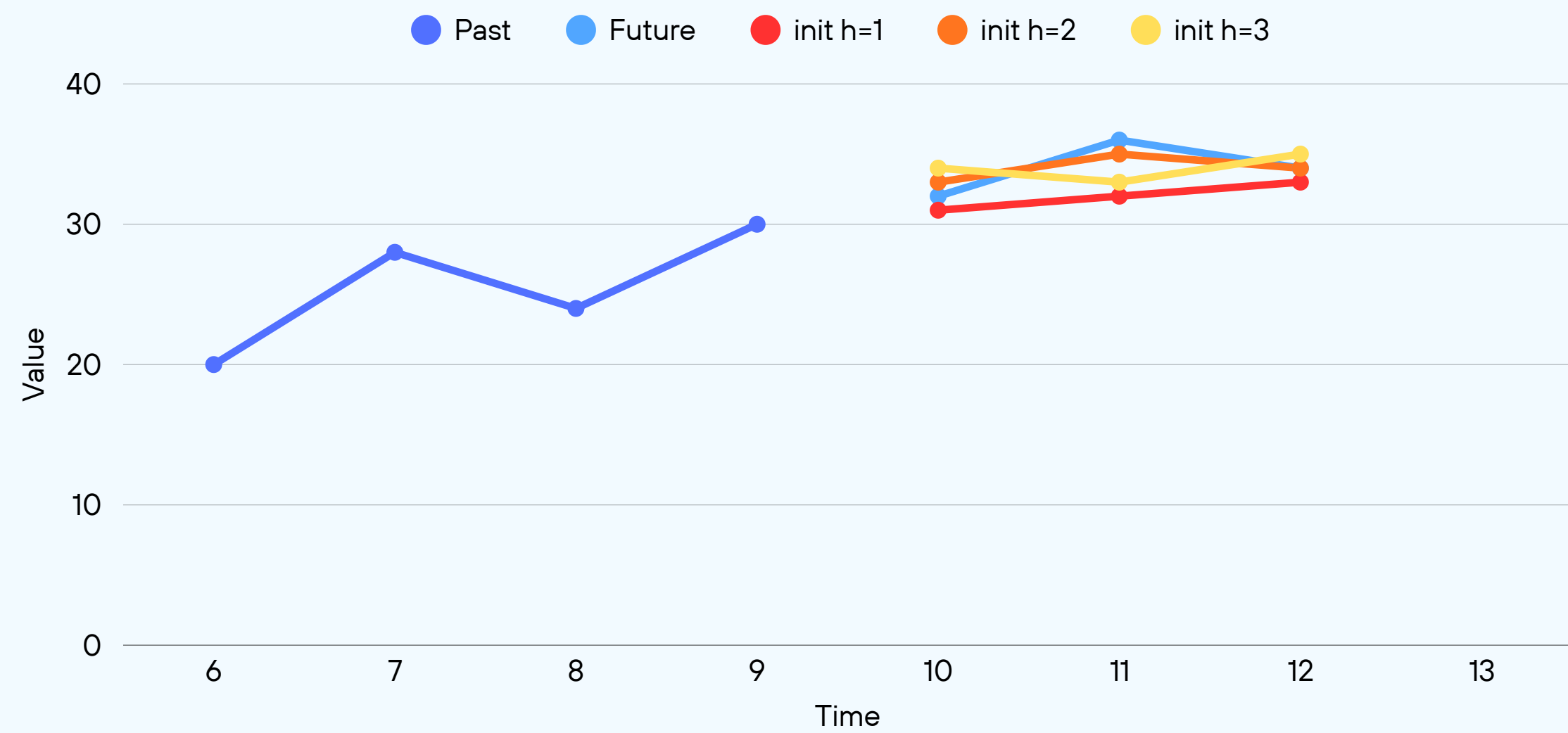
Train multiple direct models each of whom with a different forecasting horizon to predict the same time series (up to its horizon, in the example indicated with h)



02

Step 2

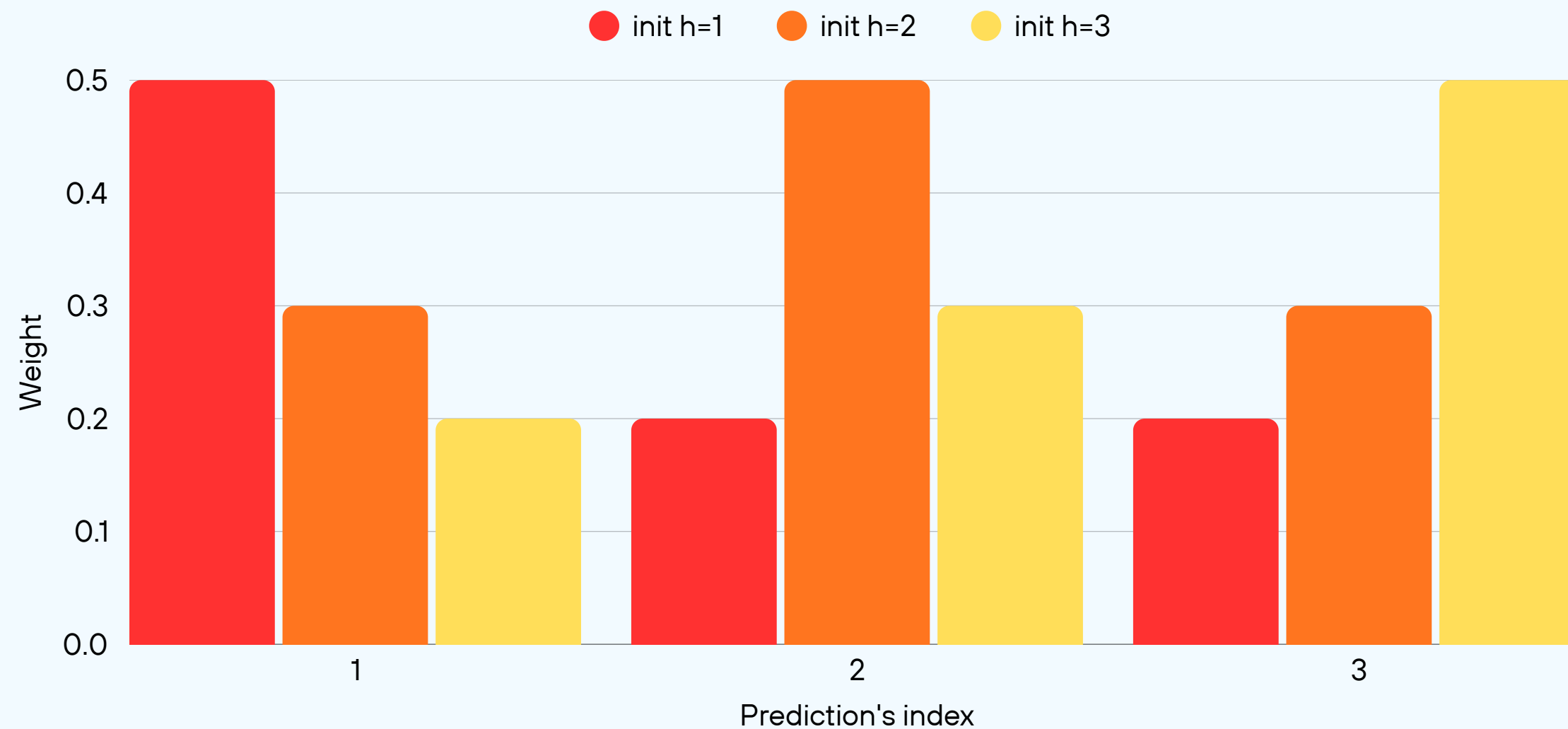
Using an autoregressive mechanism makes such that each models reaches the same desired horizon (in the example $h = 3$).



02

Step 3

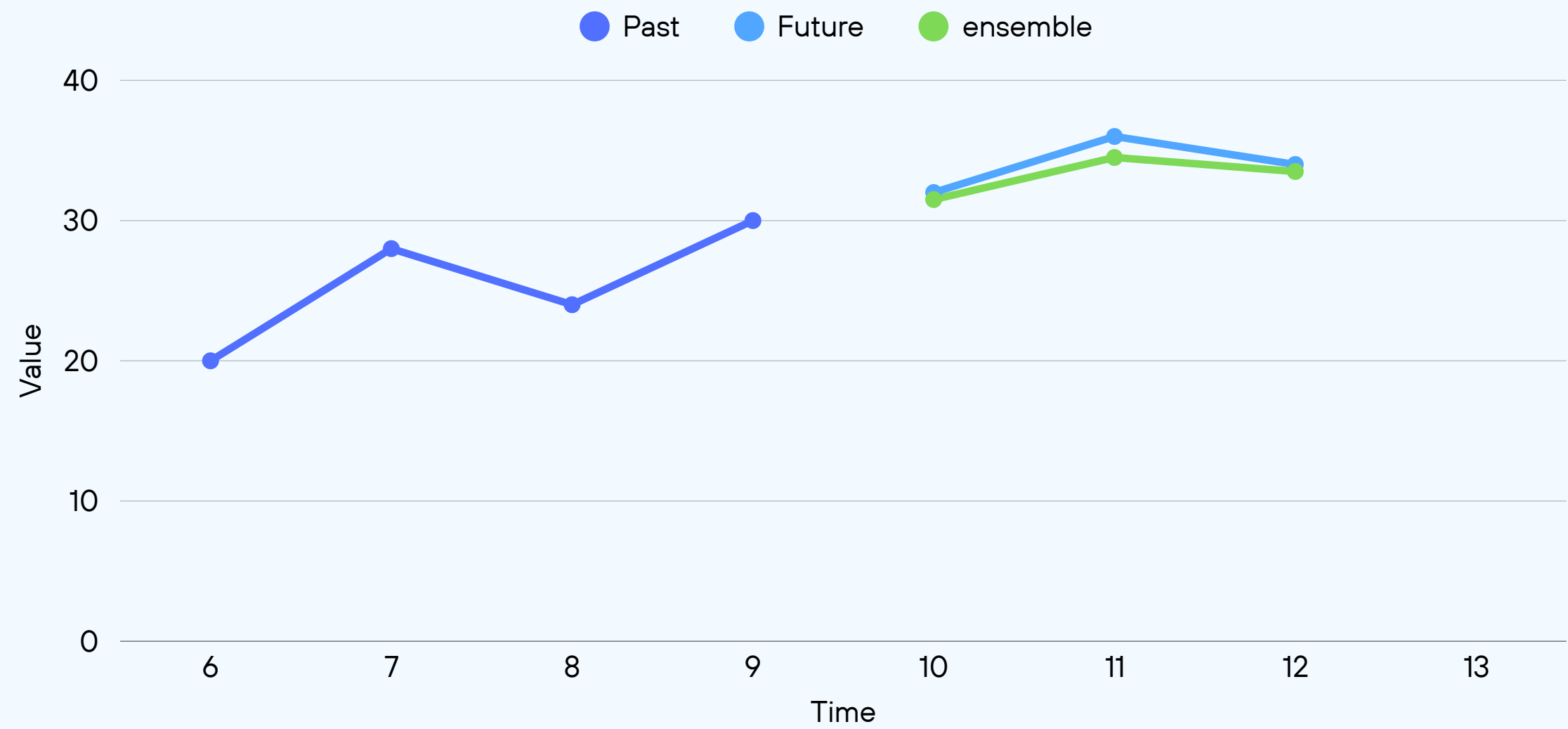
Compute for each predicted point for each model a corresponding weight based on the prediction's error variance. Normalize the weights of different models across the same index.



02

Step 4

Combine the predictions from each models that share the same index by weighted sum. This leads to the final ensemble’s prediction.



02

Framework formulation

- formal definition

Letting $X = \{x_0, \dots, x_t\}$ be a univariate time series. The goal is to predict

$$\hat{X}_f = \{\hat{x}_{t+1}, \dots, \hat{x}_{t+h}\}$$

Direct Forecasting Models

Let M_k denote a direct forecasting model with horizon k

$$M_k : \mathbb{R}^w \rightarrow \mathbb{R}^k$$

Where w is the input window's size



Autoregressive Forecasting Operator

Define an autoregressive operator $A_{h(\cdot)}$ that extends a direct forecasting model to a target horizon h by recursively feeding predictions back as inputs

$$A_{h(M_k)}(X_{t-w+1:t}) = \hat{X}_f^{(k)} \in \mathbb{R}^h$$

Model Ensemble Construction

Instantiate a family of direct models with increasing horizons

$$\mathbb{M}_h = \{M_1, M_2, \dots, M_h\}$$

Applying the autoregressive operator to each model yields

$$\mathbb{F}_h = \{A_{h(M_1)}, A_{h(M_2)}, \dots, A_{h(M_h)}\}$$

Each $A_{h(M_i)}$ produces a full forecast vector

$$\hat{X}_f^{(i)} = (\hat{x}_{t+1}^{(i)}, \dots, \hat{x}_{t+h}^{(i)})$$

Prediction Variance Estimation

For each model $i \in \{1, \dots, h\}$ and forecast step $f \in \{1, \dots, h\}$ estimate the prediction's error variance

$$v_{i,f} = \text{Var}(\epsilon_{t+f}^{(i)})$$

using an appropriate uncertainty estimation method

Variance-Weighted Aggregation

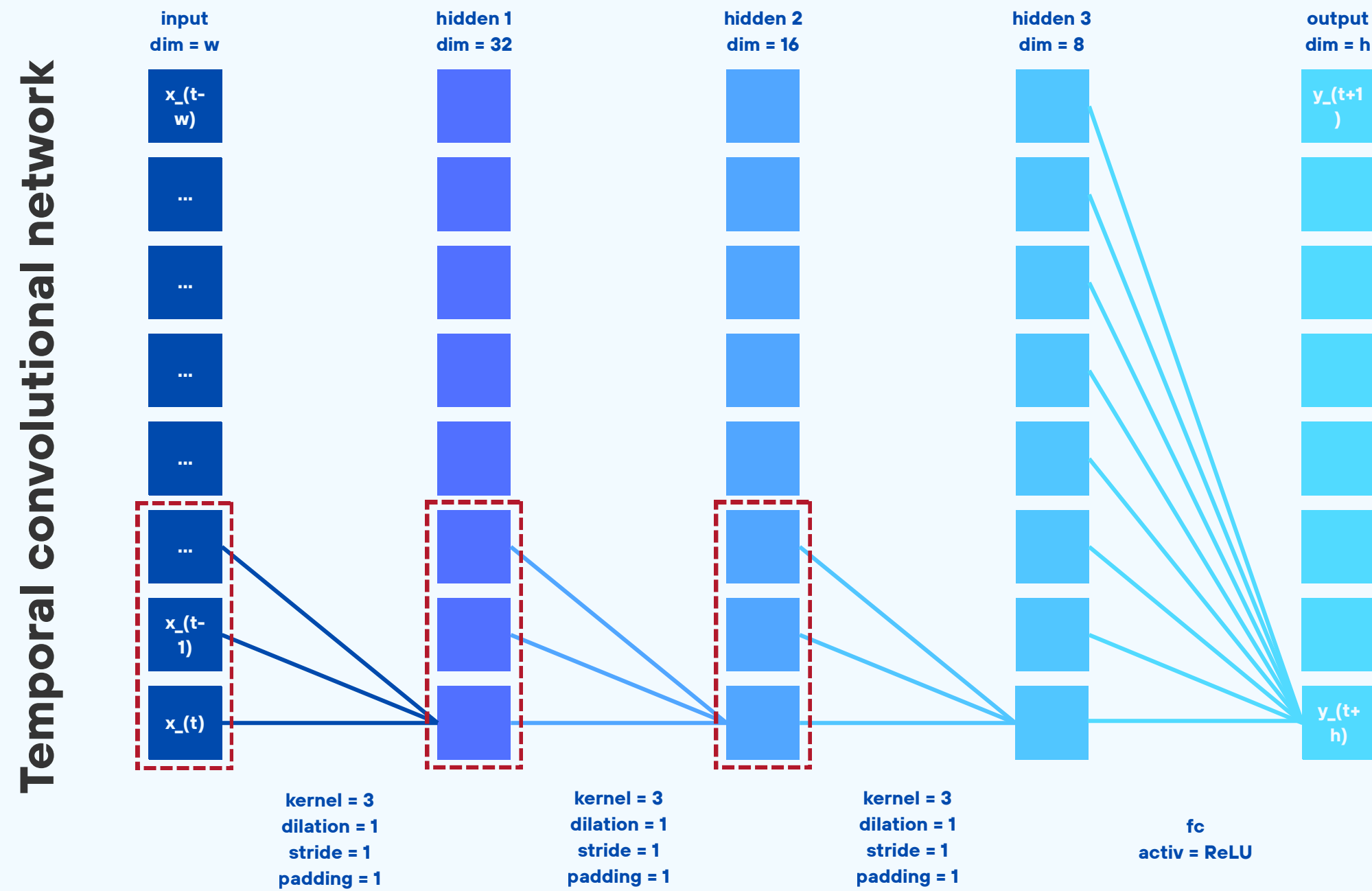
The final forecast at horizon step f is obtained via inverse-variance weighting

$$\hat{x}_{t+f} = \frac{\sum_{i=1}^h \frac{1}{v_{i,f}} \hat{x}_{t+f}^{(i)}}{\sum_{i=1}^h \frac{1}{v_{i,f}}}$$

This produces the final prediction vector

$$\hat{X}_f = \{\hat{x}_{t+1}, \dots, \hat{x}_{t+h}\}$$

Base-model choice



Horizon-aware loss

Instead of computing the usual MSE we used a penalized Huber Loss.
The penalty introduced is designed to don't let the model learning only how to predict short horizon.

Prediction's error

$$e_f = \hat{x}_{t+f} - x_{t+f}$$

Huber loss

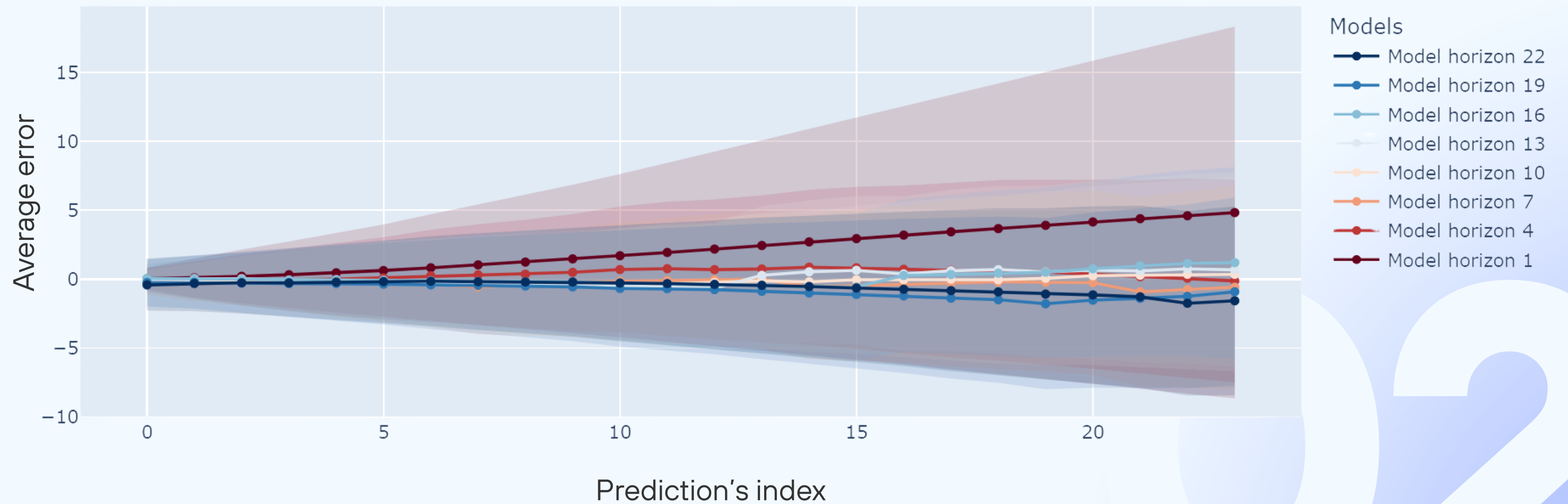
$$\mathcal{L}_\delta(e_f) = \begin{cases} \frac{1}{2}e_f^2, & \text{if } |e_f| \leq \delta \\ \delta \left(|e_f| - \frac{1}{2}\delta \right), & \text{otherwise.} \end{cases}$$

Horizon-aware

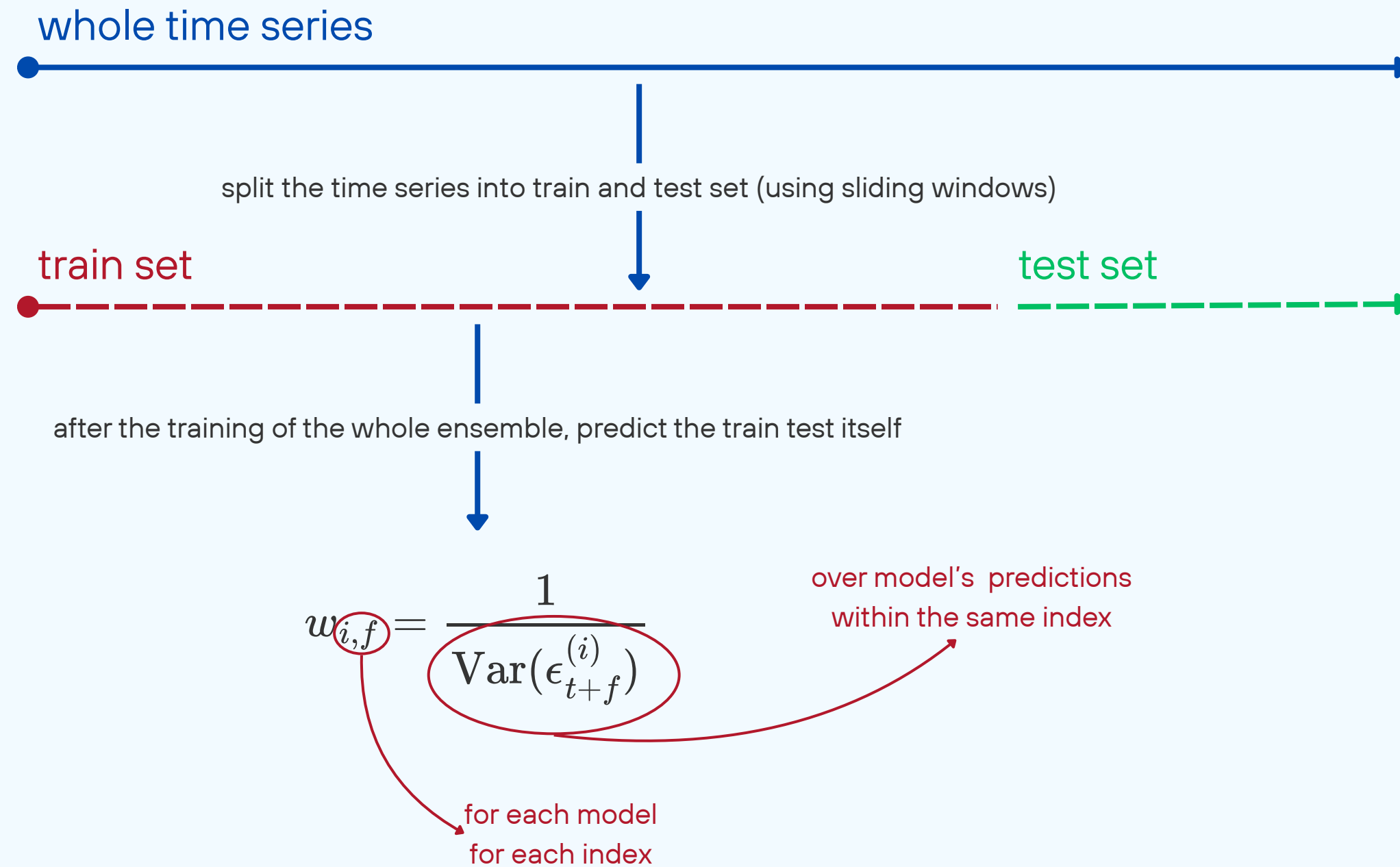
$$\mathcal{L} = \sum_{f=1}^h w_f \mathcal{L}_\delta(e_f)$$

02

Horizon-aware loss



Train and weights computation



■ **Benchmarks
Overview**

■ **Benchmarks
Details**

Evaluation

■ **Case Study**

■ **Results**

03

Benchmarks Overview

03



Electricity

Energy consumption values measured across 9 months.

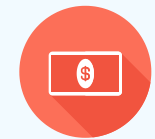
Displays **high daily seasonality**; measures vary significantly in **magnitude**.



Traffic

15 months of daily measures for occupancy rate of car lanes in San Francisco.

Features **high daily seasonality** and peak-hour traffic spikes.



Volatility

Daily realized volatility computed from the intraday data of stock indices.

Appears **noisy** with no definite seasonality.



Solar

Solar power generation values in american power plants.

Exhibits **daily seasonality** and intermittent periods of zero power production.



Wind

Percentage of wind power output across european countries.

Extremely **noisy** with very **slight yearly and monthly seasonality**.

Source:

Evaluation of Multi-step Forecasting Models

Benchmarks Details

	Number of time series	Avg length (per series)	observations delay
 Electricity	369 clients	5957	1 hour
 Traffic	963 sensors	4151	1 hour
 Volatility	31 indeces	4898	1 day
 Solar	137 plants	52560	10 minutes*
 Wind	28 sensors	10957	1 day

03

A Case Study: Solar Predictions

03

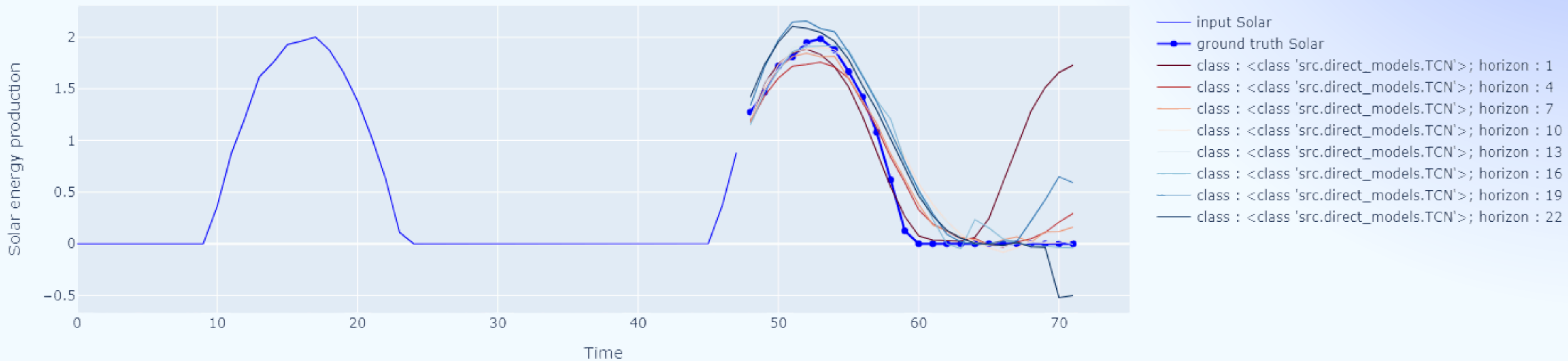
min mse : 0.02664

max mse : 0.46589

average mse : 0.10785

ensemble mse : 0.04013

Solar energy production over time



A Case Study: Solar Predictions

min mse : 0.02664

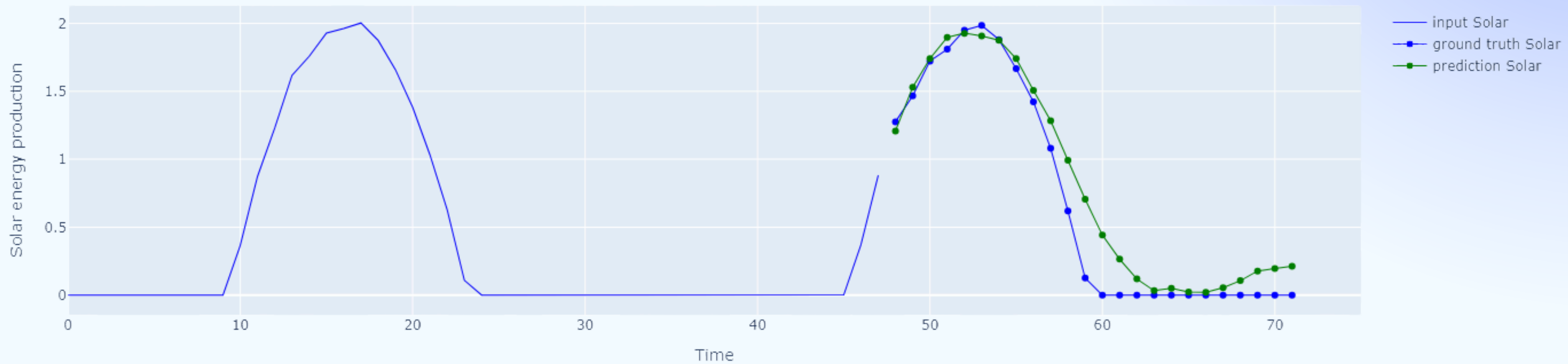
max mse : 0.46589

average mse : 0.10785

ensemble mse : 0.04013

03

Solar energy production over time



A Note on UMHEMe Train Sets



Time series are used to create sliding windows, which are the base for the training of our models.

We decided to use these windows to train UMHEMe in two different ways:

- **umheme_each:** a different ensemble is trained for each time series in the given dataset
- **umheme_all:** all time series in a dataset are merged into a single series for training

These two models are then compared against **xgboost** and **arima**. Results are shown in the next slide.

Results: MSE

A first measure we can consider is the **Average Mean Squared Error** (MSE) computed on test predictions for each model used on each dataset:

	xgboost	arima	umheme_each	umheme_all
 Electricity	0.9765	0.3011	0.7121	0.7803
 Traffic	4.468	4.547	4.627	4.629
 Volatility	0.6870	0.6833	0.6846	NaN
 Solar	0.6569	0.9630	0.6896	0.3622
 Wind	1.101	1.108	1.092	1.021



Results: Counting “Wins”

We can rank models by counting, for each dataset, in **how many time series** a model performed better than the others:

	xgboost	arima	umheme_each	umheme_all
 Electricity	11	83	4	0
 Traffic	28	45	3	24
 Volatility	18	13	0	0
 Solar	2	0	0	98
 Wind	0	2	11	15

03

Results: Averaging Ranks

Another interesting performance measure consists in **ranking the models, for each time series, based on their performance** compared to the others, and then **averaging these ranks**:

	xgboost	arima	umheme_each	umheme_all
 Electricity	3.42	1.16	2.46	2.96
 Traffic	2.03	2.29	2.90	2.78
 Volatility	1.48	2.10	2.42	NaN
 Solar	2.01	4	2.97	1.02
 Wind	2.75	3.29	1.64	2.32

03

Thank you!

Bortolussi, Bredariol, Savorgnan



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Literature references

Books

1. *"Time Series Analysis and Its Applications"* (Shumway, Stoffer, 2016)
2. *"Applied Time Series Analysis and Forecasting with Python"* (Huang, Petukhina, 2022)
3. *"Modern Time Series Forecasting Using Python"* (Joseph, Tackes, Packt, 2024)

Papers

1. *"The Combination of Forecasts"* (Bates and Granger 1969)
2. *"Time Series Forecasting With Neural Networks: A Survey"* (Bryan Lim and Stefan Zohren, 2020)
3. *"Multi-output Ensembles for Multi-step Forecasting"* (V Cerqueira, L Torgo, 2023)
4. *"A Comparative Study of Multi-Model Ensemble Forecasting Accuracy between Equal- and Variant-Weight Techniques"*
(Xiaomin Wei, Xiaogong Sun, Jilin Sun, Jinfang Yin, Jing Sun and Chongjian Liu, 2022)
5. *"Performance metrics for multi-step forecasting measuring win-loss, seasonal variance and forecast stability: an empirical study"*
(Eivind Strøm & Odd Erik Gundersen, 2024)